Must-fix decisions before build

These corrections supersede any older wording in the detailed PRD. They are mandatory for the SIH build and must be reflected in code, environment variables, testing, and the demo runbook.

Priority	Decision	Implementation requirement
MUST-FIX 1	AYUSH differentiation	Add a dedicated Dashavidha Pariksha intake branch covering Prakriti, Vikriti, Agni, Koshtha, and Sattva.
MUST-FIX 2	Voice latency risk	Use hosted Groq Whisper API as the primary transcription path. Keep browser Web Speech API as a silent fallback. Do not add self-hosted Docker Whisper for the demo.
MUST-FIX 3	Demo deployment risk	Use hosted staging as the primary demo environment: Vercel frontend, Railway API/worker, and managed PostgreSQL. Local Docker Compose remains a development fallback only.

1. AYUSH intake branch

The intake is not generic/allopathic only. Patients may choose the AYUSH pathway when the clinic enables it. The pathway collects Dashavidha Pariksha inputs in a culturally respectful, clinician-reviewable format. It is structured intake, not autonomous diagnosis.

Section	Minimum capture	Output label
Prakriti	patient-reported constitution observations and optional clinician notes	AYUSH - Prakriti
Vikriti	current imbalance / symptom pattern as reported	AYUSH - Vikriti
Agni	appetite, digestion, regularity, and patient language	AYUSH - Agni
Koshtha	bowel pattern and relevant patient-reported context	AYUSH - Koshtha
Sattva	wellbeing, sleep, stress, and mental state prompts	AYUSH - Sattva

2. Voice and deployment architecture

Area	Primary path	Fallback / degraded mode	Acceptance test
Speech	Browser records audio -> Railway worker -> hosted Groq Whisper API	Browser Web Speech API is silent fallback; manual typing always available	Provider timeout does not block intake.
Frontend	Vercel staging deployment	Local Next.js only for development	Demo URL works before event day.
API and worker	Railway service with health/readiness checks	Local process for development	Worker retries are observable.
Database	Managed PostgreSQL	Local PostgreSQL via Docker Compose	Migration and seed run against staging.
Storage	Hosted private object storage	Local S3-compatible storage	Signed access and retention tested.

3. Required environment variables

Each teammate creates a local .env from .env.example. Real secrets stay in the hosting provider secret store or local uncommitted .env. Never commit keys.

Variable	Local/mock value	Staging value
AI_PROVIDER_MODE	mock	groq
GROQ_API_KEY	blank	Railway secret
WHISPER_PROVIDER_MODE	mock	groq-hosted
WEB_SPEECH_FALLBACK	true	true
AYUSH_INTAKE_ENABLED	true	true
DATABASE_URL	local Postgres	Railway managed Postgres URL
NEXT_PUBLIC_API_URL	http://localhost:8000	Railway API URL
JWT_SECRET	random local secret	Railway secret

4. Module ownership changes

Module	Required change
Backend lead	Add typed settings, provider adapter interface, staging deployment files, CORS for Vercel, readiness checks, and shared AYUSH enums.
Auth	Store selected intake pathway and AYUSH consent language; preserve patient privacy and role scope.
AI Summary	Summarize AYUSH fields with explicit provenance; never merge model inference into clinician-confirmed facts.
Whisper and OCR	Implement Groq Whisper adapter, browser fallback contract, retry/status states, and hosted storage configuration.
Doctor Queue	Show pathway badge in queue and doctor workspace; allow doctor to filter/review AYUSH visits without changing FIFO rules.

5. Final Antigravity instruction

Read BASE_PRD_MediKiosk.pdf first, then the assigned module PRD. Apply the three mandatory final corrections: implement the Dashavidha Pariksha AYUSH intake branch, use hosted Groq Whisper as primary with browser Web Speech API as silent fallback, and target Vercel + Railway + managed PostgreSQL staging for the demo. Do not implement self-hosted Docker Whisper as the primary demo path. Keep API keys in environment variables or hosting secrets, never source control. Work only in the assigned branch and owned files, add tests, and document the staging integration.

These changes are mandatory acceptance criteria for every module, even when the module only consumes the shared contracts.

MediKiosk

AI-enabled clinic operations for Bharat

BASE PRD | Project Bible

Shared context, product scope, architecture, and delivery contract

v1.0 | 28 Aug 2026

Implementation-ready product requirements package

Document control

Field	Value
Document	BASE_PRD_MediKiosk
Owner	Product + Backend Lead
Status	Build-ready baseline
Audience	SIH team, reviewers, Antigravity agent
Expected length	25-35 pages
Purpose	One shared source of truth for the full MediKiosk platform.

How to use this document

Read the BASE_PRD first, then the relevant module PRD. Treat API contracts, ownership boundaries, validation rules, and acceptance criteria as the source of truth. If implementation pressure requires a change, record it in the decision log and keep branch ownership intact.

1. Executive summary

MediKiosk is a multilingual, AI-assisted clinic kiosk and staff console for small hospitals and primary health centers. It reduces registration friction, converts spoken symptoms and uploaded records into structured clinical context, and gives doctors a safe queue-aware workspace. The product is an assistive workflow system: it does not diagnose, prescribe, or replace clinician judgment.

The SIH opportunity is to combine accessible kiosk interaction, Indian-language speech, OCR for paper records, structured summaries, and queue transparency in one deployable architecture. The first release optimizes for a reliable demo and a defensible production foundation: explicit consent, human review, traceability, and predictable APIs.

North-star outcome: a patient can arrive with minimal digital literacy, complete intake in a few minutes, and hand a doctor a concise, reviewable context packet.

2. Problem statement

Problem	Current impact	MediKiosk response
Long queues and repeated forms	Patients repeat the same story to reception and clinician.	Digital intake with token-based queue and reusable patient profile.
Language and literacy barriers	English-only forms cause omissions and staff rework.	Voice-first intake, language selection, simple prompts, and fallback typing.
Paper-heavy history	Prior prescriptions and reports are hard to search.	Secure uploads, OCR extraction, and timeline-linked history.
AI trust gap	Opaque AI output can mislead or be ignored.	Evidence-linked summary, confidence labels, human review, and audit trail.
Disconnected operations	Front desk, doctor, and patient lack one live state.	Shared status model, queue events, and role-based workspaces.

3. Vision, goals, and non-goals

Vision: make every clinic visit understandable, organized, and clinically reviewable, even when the patient brings voice notes and paper documents instead of a polished digital record.

Goals for the SIH build

• Demonstrate an end-to-end patient-to-doctor journey in under ten minutes.

• Support English plus a configurable Indian-language pack, with Hindi as the initial demonstration language.

• Produce a structured AI summary that clearly separates patient-reported facts, extracted document facts, and model-generated suggestions.

• Keep every sensitive operation attributable to a user, role, timestamp, and request ID.

• Make module ownership explicit so five contributors can work in parallel and merge safely.

Non-goals

• Autonomous diagnosis, triage, prescription, or emergency decision-making.

• Replacing an existing hospital information system in the SIH prototype.

• Billing, insurance adjudication, pharmacy inventory, or lab device integration in v1.

• Training a custom medical model; v1 uses provider APIs behind an adapter interface.

4. Personas and permissions

Persona	Needs	Key actions	Permissions
Patient	Fast, private, language-accessible intake.	Register/find profile, consent, record symptoms, upload documents, view token/status.	Own profile, own visit inputs, own status.
Receptionist	Move people through intake reliably.	Verify identity, create visit, issue token, correct demographic errors.	Patient lookup, queue write, no clinical summary editing.
Doctor	Review context quickly and decide clinically.	Open queue, review summary and source evidence, add notes, complete visit.	Clinical view, summary feedback, visit disposition.
Clinic admin	Keep clinic configuration safe.	Manage staff, languages, queue settings, retention policy.	Configuration and audit read.
System operator	Keep service healthy.	Monitor health, retries, failures, storage, and integration keys.	Operational metadata only; no routine clinical content.

5. Complete patient journey

A. Arrival and identity: the patient chooses language, reads a privacy notice, and either creates a profile with phone verification or asks reception to locate an existing record. A visit is then created for the selected clinic.

B. Consent and intake: the patient explicitly consents to processing and selects input mode: speak, type, or upload. Audio is transcribed, documents are OCR-processed, and raw inputs remain linked to the visit.

C. Review and queue: the kiosk shows a plain-language review of captured facts. The patient can correct or remove an item. Submission creates a queue entry and token; the patient sees approximate position and a neutral waiting message.

D. Doctor review: the doctor sees queue status, patient-reported facts, extracted facts with source references, medical-history timeline, and a model summary with uncertainty labels. The doctor confirms, edits, or rejects content before using it.

E. Closure: doctor records disposition, follow-up recommendation, and optional note. The visit becomes closed; patient sees only the approved patient-facing status, not internal reasoning or sensitive staff notes.

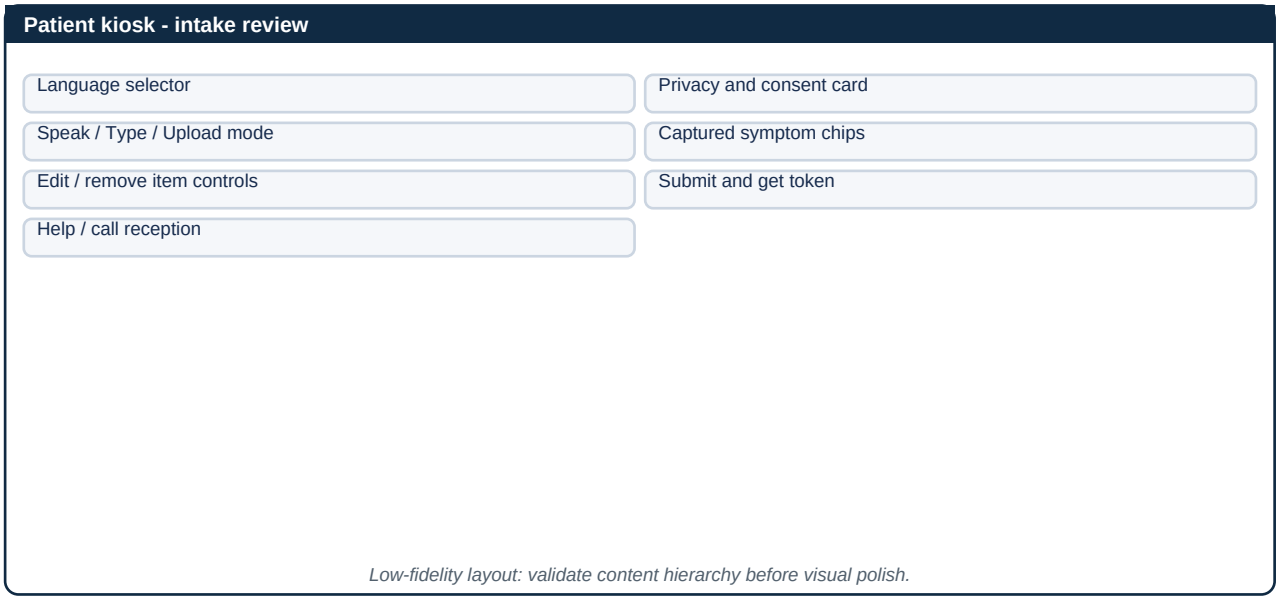


Figure 1. Patient-facing hierarchy prioritizes comprehension, correction, and consent.

6. Doctor workflow

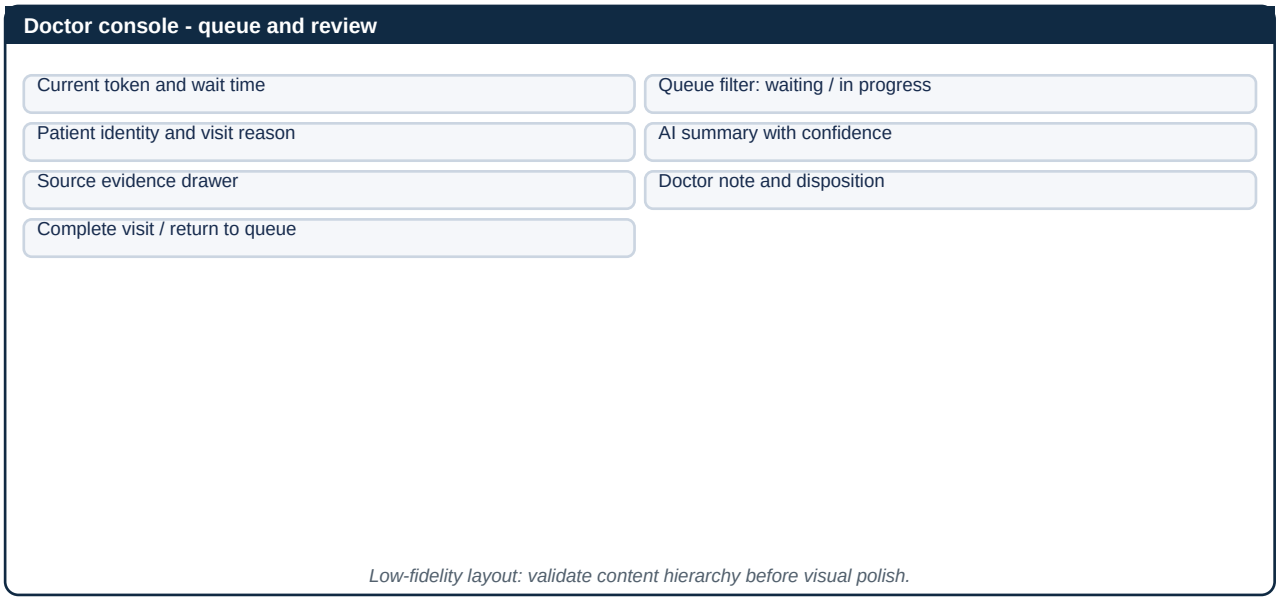


Figure 2. Doctor workspace keeps queue context visible while preserving evidence access.

Queue state machine

State	Owner	Allowed transitions	Meaning
WAITING	Reception/system	CALLED, CANCELLED	Patient has a valid token.
CALLED	Reception/doctor	IN_PROGRESS, NO_SHOW	Patient is being requested.
IN_PROGRESS	Doctor	COMPLETED, NEEDS_REVIEW	Clinical work is active.
NEEDS_REVIEW	Doctor	COMPLETED	Missing confirmation or source review.
COMPLETED	Doctor	-	Visit closed and immutable except correction workflow.
CANCELLED / NO_SHOW	Reception	-	No clinical completion; retained for audit.

7. Functional requirements

ID	Requirement	Priority	Acceptance signal
FR-01	Patient can select language and complete consent before processing.	Must	No AI or upload processing occurs without consent.
FR-02	Patient can register or be found by verified identifier.	Must	Duplicate detection and clear recovery path.
FR-03	Visit creation issues a unique clinic-scoped token.	Must	Token is never duplicated on concurrent requests.
FR-04	Audio is transcribed and linked to visit.	Must	Raw audio and transcript have status and trace.
FR-05	Image/PDF uploads are stored securely and OCR is asynchronous.	Must	Unsupported/oversize files fail with actionable status.
FR-06	AI summary returns structured facts and uncertainty.	Must	Schema validation rejects malformed provider output.
FR-07	Doctor can inspect evidence and edit final note.	Must	AI output is never silently treated as clinician-authored.
FR-08	Queue updates are visible without page refresh.	Should	Polling or websocket adapter keeps status within 5 seconds.
FR-09	Audit log records sensitive actions.	Must	Actor, action, entity, request ID, and timestamp retained.
FR-10	Admin can configure clinic languages and queue policy.	Should	Configuration is role-protected and versioned.

8. Non-functional requirements

Area	Target
Availability	99% for demo deployment; graceful degraded mode when AI provider is unavailable.
Performance	p95 read APIs under 500 ms excluding AI; queue refresh under 5 seconds.
Security	TLS, hashed passwords, short-lived access tokens, refresh rotation, least privilege, upload scanning hook.
Privacy	Consent-gated processing, configurable retention, no sensitive content in logs, redacted analytics.
Accessibility	Keyboard support for staff UI, high contrast, 16 px minimum body text, plain-language errors.
Reliability	Idempotency keys for visit creation and uploads; retries with backoff for provider calls.
Observability	Structured logs, request ID, health/readiness endpoints, provider latency and failure counters.
Portability	Docker Compose for local demo; PostgreSQL-compatible SQL; object storage via S3-compatible API.

9. System architecture

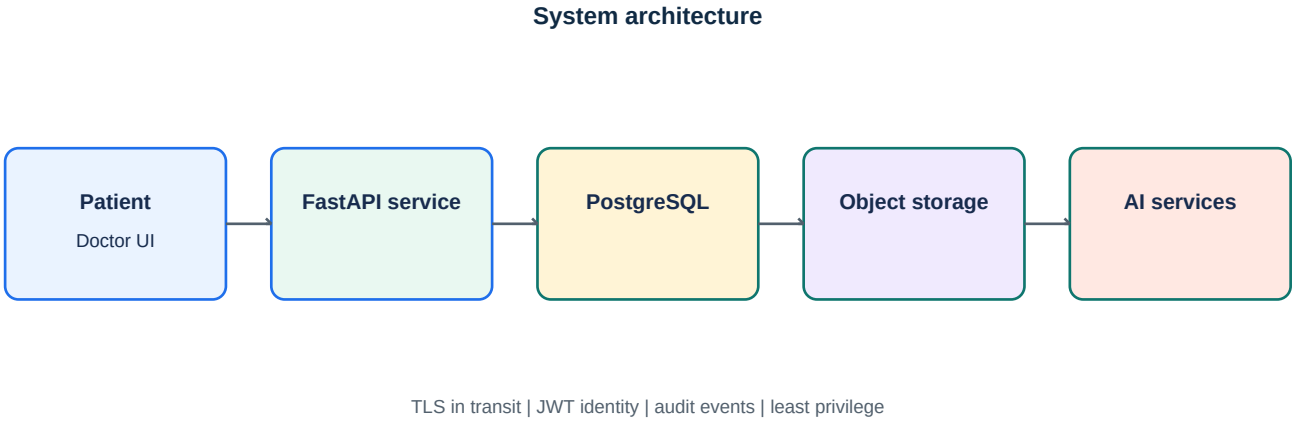


Figure 3. Logical architecture. The service layer owns policy and data contracts; external AI providers are adapters.

Request path

Browser or kiosk client -> FastAPI router -> auth and policy dependencies -> service layer -> repository / provider adapter -> PostgreSQL or object storage. Long-running transcription, OCR, and summarization should execute through a job boundary even if the demo uses an in-process worker.

AI pipeline

Stage	Input	Output	Safety boundary
1. Capture	Audio, typed text, image/PDF	Raw object + visit input record	Consent, size/type validation, malware scan hook.
2. Transcribe	Audio object	Transcript + segments + provider metadata	Language, confidence, retry, raw source retained.
3. Extract	Document object	OCR text + page references	Do not invent missing text; mark unreadable regions.
4. Normalize	Transcript/OCR/typed facts	Canonical visit facts	Schema validation and provenance per fact.
5. Summarize	Canonical facts + history	Structured summary JSON	Prompt constraints, low temperature, JSON schema, retry.
6. Review	Summary + sources	Doctor-confirmed note	Human approval required before clinical use.

10. Data model and ER diagram

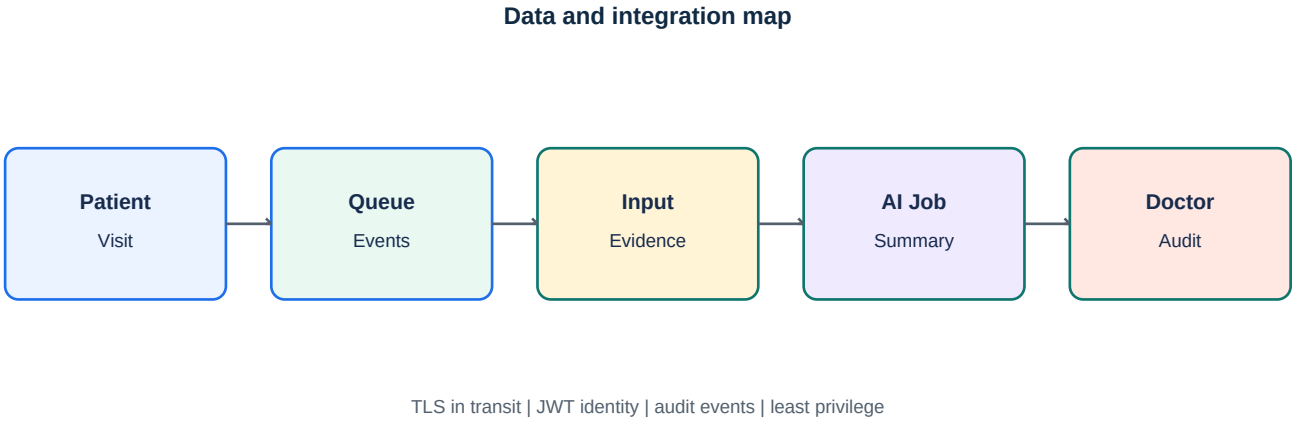


Figure 4. Domain relationships are anchored on Patient and Visit; every AI output is visit-scoped and traceable.

Core schema

Table	Important fields	Indexes / constraints
patients	id, phone_hash, name, dob, language, created_at	unique phone_hash; normalized phone; soft delete.
users	id, role, clinic_id, email, password_hash, is_active	unique email; clinic role constraint.
visits	id, patient_id, clinic_id, status, token, consent_at, created_by	unique clinic_id + token + service_date; status index.
queue_entries	id, visit_id, position, state, called_at	unique visit_id; clinic/state index.
visit_inputs	id, visit_id, kind, object_key, text, status, provenance	visit/kind index; immutable raw reference.
ai_jobs	id, visit_id, type, status, attempts, provider, error_code	status/created index; idempotency key.
summaries	id, visit_id, version, payload_json, confidence, reviewed_by	unique visit_id + version; review index.
audit_events	id, actor_id, action, entity_type, entity_id, request_id, payload_json	request and entity indexes; append-only.

11. API standards

Base URL: /api/v1. JSON is UTF-8. All timestamps are ISO 8601 UTC. Authentication uses Authorization: Bearer . Mutating endpoints accept X-Request-ID; idempotent endpoints also accept Idempotency-Key.

Success envelope

```
{
  "success": true,
  "data": { "id": "uuid", "status": "WAITING" },
  "meta": { "request_id": "req_123" },
  "error": null
}
```

Error envelope

```
{
  "success": false,
  "data": null,
  "meta": { "request_id": "req_123" },
  "error": { "code": "VALIDATION_ERROR", "message": "Readable message", "fields": {} }
}
```

Rule	Decision
Validation	Pydantic at edge; business rules in service layer; database constraints as final guard.
Pagination	limit + cursor for lists; max limit 100; stable created_at/id ordering.
Errors	Stable machine code, human message, optional fields map; never provider raw secrets.
Versioning	/api/v1; additive changes preferred; breaking changes require /v2 and migration note.
OpenAPI	Every route tagged by module with examples and auth requirements.

12. Repository and Git workflow

Reference layout

```
apps/
  api/app/
    api/            # routers and dependencies
    core/           # settings, security, logging
    db/             # session, models, migrations
    schemas/        # Pydantic request/response models
    services/       # domain use cases
    repositories/   # persistence queries
    integrations/   # AI, storage, messaging adapters
    tests/
  web/              # minimal test frontends, not the final kiosk UI
docs/
  prd/
infra/
  docker-compose.yml
```

Branch ownership

Branch	Owner	Scope
main	Backend lead	Architecture, shared DB, integration, merge fixes.
feature/auth	Member 1	Registration, login, patient profile APIs, auth test UI.
feature/ai-summary	Member 2	Summary jobs, prompt adapter, history, review UI.
feature/ocr-whisper	Member 3	Audio, OCR, upload/storage adapters, processing UI.
feature/doctor-queue	Member 4	Queue, doctor workspace APIs, queue test UI.

Merge policy: small commits, no unrelated formatting, tests required, rebase before handoff, PR description includes API changes and screenshots, main owner resolves shared-file conflicts.

13. UI and content guidelines

Principle	Application
Calm clinical utility	White space, navy structure, teal success, amber attention; avoid alarmist red except destructive/error states.
One task per screen	The kiosk should ask one clear question at a time; staff screens may use split panes.
Explain AI	Use labels such as Patient said, Document says, Model summary, Doctor confirmed.
Make correction normal	Every captured item has edit/remove; never make users defend a correction.
Low-bandwidth ready	Compress media, show progress, retry without losing state, avoid heavy animation.
Inclusive language	No jargon, no hidden defaults, no shame language for literacy or symptoms.

14. Acceptance and demo script

Step	Expected result
1. Start services	Health and readiness pass; seed clinic is visible.
2. Register patient	Consent captured; patient created; duplicate path tested.
3. Capture voice + document	Jobs created; transcript/OCR status progresses; evidence is linked.
4. Create visit	Token generated once; queue position visible.
5. Doctor reviews	Summary shows facts, provenance, uncertainty, and editable note.
6. Complete visit	Visit closes; audit event exists; patient receives approved status.
7. Simulate provider failure	User sees retry/degraded message; no silent data loss.

15. Risks and mitigations

Risk	Mitigation
AI hallucination	Strict JSON schema, provenance, human review, explicit uncertainty, prompt tests.
Poor audio / OCR	Language selector, retry, manual typing, readable error, source preview.
PII leakage	Redacted logs, secrets manager, retention policy, access checks, audit.
Merge conflicts	Owned files, shared contract freeze, small commits, integration branch checks.
Demo fragility	Seed data, provider mocks, fixture media, degraded mode, deterministic happy path.

16. Glossary

Term	Meaning
Visit	One clinic encounter for one patient.
Evidence	Raw or extracted input supporting a fact or summary item.
AI job	Asynchronous unit of transcription, OCR, normalization, or summarization.
Queue token	Clinic-scoped human-readable position identifier for a visit.
Provenance	Metadata describing where a fact came from and how confident it is.
Degraded mode	Safe operation when an external AI/storage dependency is unavailable.

17. Final integration contract

All modules must expose typed service functions, route-level schemas, predictable error codes, database migrations, tests, and a minimal test page. A module is integrated only when it can be run from a clean checkout, its endpoints appear in Swagger, its happy path is demonstrated with seeded data, and its failure paths are observable.

Antigravity prompt

Read BASE_PRD_MediKiosk.pdf first. Treat it as the source of truth for product scope, architecture, privacy, API conventions, database concepts, Git ownership, and acceptance criteria. Then read the assigned module PRD. Build only the assigned module in its branch, use FastAPI + SQLAlchemy + Pydantic conventions, create a minimal Next.js testing frontend only for the module APIs, do not rewrite unrelated modules, and leave the repository runnable with migrations, tests, examples, and documented integration notes.

18. Detailed field dictionary

Field	Type	Rule	Example
patient.language	enum	Required; allow configured clinic languages	hi
visit.consent_at	UTC datetime	Required before processing	2026-08-28T10:15:00Z
visit.status	enum	State transition only through service	IN_PROGRESS
input.provenance	object	Source and confidence required	{"source":"transcript","confidence":0.86}
summary.version	integer	Monotonic per visit	2

19. Threat model

Threat	Attack surface	Control	Verification
Account takeover	Login and refresh tokens	Hashing, expiry, rotation, rate limits	Auth negative tests
Cross-patient access	IDs in URL and object keys	Policy dependency plus repository scope	Authorization matrix
Prompt injection	Uploaded document or transcript	Treat inputs as data; fixed system prompt; output schema	Adversarial fixtures
Data leakage	Logs, traces, exports	Redaction and allowlisted metadata	Log inspection
Malicious upload	File parser/provider	Type/size check, scan hook, isolated worker	EICAR-like fixture
Queue race	Concurrent claim/token requests	DB transaction and row lock	Concurrency test

20. Observability contract

Signal	Required fields	Dashboard question
Request log	request_id, route, status, latency, actor_role	Which endpoint is slow or failing?
AI job metric	job_type, provider, attempt, duration, outcome	Are provider failures increasing?
Queue metric	clinic, waiting_count, oldest_wait_minutes	Is the clinic moving patients?
Audit event	actor, action, entity, timestamp	Who changed sensitive state?
Readiness	dependency, status, checked_at	Can the demo accept traffic?

Never log raw audio, document text, password, access token, or full patient profile. Use IDs and redacted summaries.

21. Localization and accessibility specification

• All user-visible strings live in a locale dictionary, not route code.

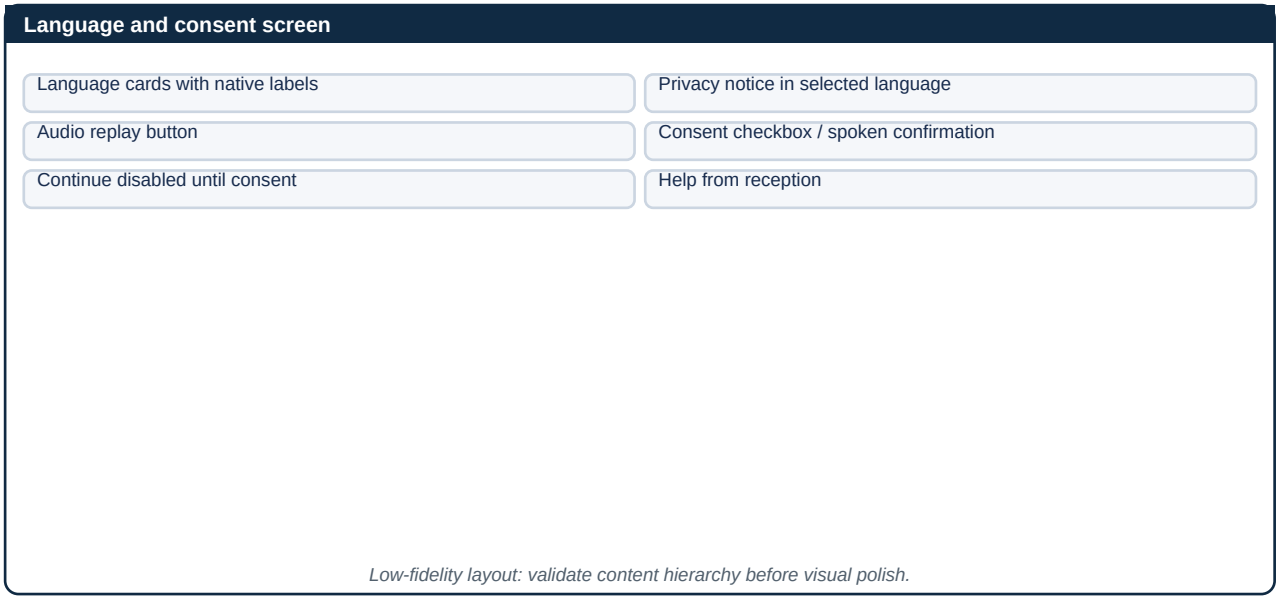
• The initial demo supports English and Hindi; fallback language is English.

• Audio prompts are short, confirm the action, and offer replay.

• Every error has a spoken-friendly message and a visual message.

• Staff keyboard order follows reading order; focus is visible; destructive actions require confirmation.

• Use high contrast and do not rely on color alone for queue state.



22. Provider adapter contract

Adapter boundary

```
class SpeechProvider(Protocol):
    async def transcribe(self, object_key: str, language: str) -> TranscriptResult: ...

class OCRProvider(Protocol):
    async def extract(self, object_key: str) -> OCRResult: ...

class SummaryProvider(Protocol):
    async def summarize(self, evidence: EvidencePacket) -> SummaryResult: ...
```

Adapters return typed results and normalized error categories. No route imports an SDK directly. Mocks implement the same protocol for deterministic demos.

23. Data retention and deletion

Data	Default retention	Deletion rule
Raw audio	30 days after visit closure	Delete object and metadata together.
Uploaded document	90 days after closure	Retain only if clinic policy requires.
Transcript/OCR	Until visit deletion	Delete with source or mark unavailable.
Summary versions	Until visit deletion	Keep audit trail of review decision.
Audit metadata	12 months minimum for demo	Never include clinical payload by default.

Retention values are configurable by clinic policy and must be documented in deployment settings.

24. Error taxonomy

Code	HTTP	User meaning	Operator meaning
VALIDATION_ERROR	422	Please correct highlighted fields.	Client sent invalid shape.
UNAUTHORIZED	401	Please sign in again.	Missing or expired token.
FORBIDDEN	403	You do not have access.	Policy denied.
CONFLICT	409	This action is already complete or changed.	State or idempotency conflict.
DEPENDENCY_UNAVAILABLE	503	Processing is delayed; retry shortly.	Provider/storage readiness issue.
INTERNAL_ERROR	500	Something went wrong; ask staff.	Unexpected bug; request ID required.

25. Release and rollback plan

• Tag a demo release only after clean migration, seed, test, and walkthrough.

• Apply additive migrations first; deploy code second; remove old fields last.

• Keep provider adapters behind feature flags so mocks can run offline.

• Rollback code without rolling back a migration unless the migration is proven safe.

• If an AI job schema changes, keep parser compatibility for one prior version.

• Capture a screenshot and request ID for every failed demo step.

26. Scenario matrix

Scenario	Expected system behavior	Owner
No consent	Input processing is blocked with clear explanation.	Auth + media
Duplicate patient	Offer sign-in/verify path; do not silently create second profile.	Auth
Unreadable document	Store source, mark OCR partial, invite manual entry.	OCR
Provider timeout	Job retries with bounded attempts and visible status.	AI / OCR
Doctor refresh	State remains correct; no duplicate claim.	Queue
Clinic closure	New visit creation is blocked with configured message.	Backend

27. SIH demonstration narrative

Asha arrives at a PHC, selects Hindi, gives consent, speaks about fever and cough, and uploads an old prescription. MediKiosk transcribes the voice, extracts readable medicine text, creates token A12, and shows a calm waiting state. The doctor claims A12, sees the patient's words and evidence links, reviews the AI summary, corrects one item, and completes the visit. The evaluator can then see the audit trail and simulate a provider outage to show safe degradation.

28. Open questions to resolve before production

Question	Decision owner	Default for SIH
Which Indian languages are live?	Product	English + Hindi
Which identity verification is acceptable?	Clinic admin	Phone OTP stub / staff verification
Which storage provider is approved?	Operator	S3-compatible local storage
What is the medical escalation path?	Clinical advisor	Display emergency disclaimer and call staff
How are deletion requests handled?	Product + legal	Manual admin workflow documented

29. Review checklist for evaluators

- Can a patient complete intake without staff explaining every field?
- Can a doctor distinguish source facts from model output?
- Can the team recover when a provider fails?
- Can the team show who approved a summary?
- Can a contributor identify exactly which files they own?
- Can the product be deployed with secrets separated from source?

30. BASE PRD sign-off

Role	Sign-off statement
Product lead	Scope and user journeys are understood.
Clinical advisor	Assistive boundaries and review requirements are acceptable.
Backend lead	Contracts, schema, and integration seams are implementable.
Security reviewer	Consent, authorization, storage, and logging controls are testable.
Demo lead	Happy path and failure path can be demonstrated.

Environment and API key setup

Each teammate creates a local environment file after checking out their branch. Never paste real keys into a PRD, source file, GitHub issue, screenshot, or committed .env file. Commit only .env.example with placeholder values.

Setup steps

• From the repository root, copy .env.example to .env.

• Fill the values locally. Use the team's approved provider accounts or local mock mode for development.

• Start the database, object storage, and API using the documented Docker Compose command.

• Open /health and /ready, then open /docs to confirm the API is configured.

• If a key is rotated, update only local secrets or the team secret store; never amend old commits to hide a leaked key.

• Before pushing, verify .env is listed in .gitignore and run the repository secret scan.

Recommended .env.example

Safe placeholder file

```
APP_ENV=local
DATABASE_URL=postgresql+psycopg://medikiosk:medikiosk@localhost:5432/medikiosk
JWT_SECRET=replace-with-a-long-random-local-secret
JWT_EXPIRE_MINUTES=30
STORAGE_ENDPOINT=http://localhost:9000
STORAGE_ACCESS_KEY=local-access-key
STORAGE_SECRET_KEY=local-secret-key
STORAGE_BUCKET=medikiosk-private
AI_PROVIDER_MODE=mock
OPENAI_API_KEY=
WHISPER_PROVIDER_MODE=mock
OCR_PROVIDER_MODE=mock
CORS_ORIGINS=http://localhost:3000
LOG_LEVEL=INFO
```

Which values are real secrets?

Variable	Purpose	Who provides it	Safe local default
OPENAI_API_KEY	GPT summary provider	Backend lead / approved account	Blank when AI_PROVIDER_MODE=mock
STORAGE_SECRET_KEY	Private object storage	Operator	Local MinIO secret
JWT_SECRET	Signs access tokens	Backend lead	Generated random local value
DATABASE_URL	Database credentials	Backend lead	Local Docker database
OCR / Whisper key	Speech or OCR provider	Media module owner	Blank when provider mode is mock

Branch rule: every teammate uses their own .env on their own branch checkout. The same variable names must work across branches. If a new variable is needed, add its placeholder to .env.example and tell the backend lead before merging.

Shared implementation checklist

Check	Definition of done
Local setup	README, .env.example, migrations, seed data, and one-command startup documented.
API quality	OpenAPI tags, consistent envelope, validation errors, auth dependencies, and correlation IDs.
Data quality	UTC timestamps, foreign keys, indexes for queue/history lookups, and soft-delete policy.
Security	Secrets outside source, password hashing, JWT expiry, upload limits, and audit events.
Demo readiness	Seeded clinic, test users, sample transcript/document, and a happy-path script.

Decision log template

Date	Decision	Reason	Owner
YYYY-MM-DD	Short decision statement	Tradeoff or constraint	Name / role
YYYY-MM-DD	Short decision statement	Tradeoff or constraint	Name / role